

Resumen técnico — Arquitecturas, patrones y organización de paquetes

Documento depurado de la conversación. Incluye definiciones, conexiones entre conceptos, comparaciones, ejemplos de estructura de paquetes, rúbrica aplicada y resultados de evaluación.

1) Glosario compacto (qué es y a qué corresponde)

Término	Qué es / A qué corresponde	Tipo / clasificación	Cuándo se usa típicamente
Monolítica	Una sola aplicación/artefacto desplegable que contiene toda la lógica	Arquitectura de software	Apps medianas/simples, time-to-market rápido
Capas	Organización por responsabilidades (presentación, aplicación, dominio, infraestructura; o 3 capas clásicas)	Patrón estructural / Patrón arquitectónico	Mantenimiento y pruebas; separación clara de responsabilidades
MVC	Separación de Modelo, Vista y Controlador	Patrón de diseño arquitectónico (UI/servidor web)	Web clásico, frameworks que lo implementan en UI o server-side
Hexagonal (Puertos y Adaptadores)	Núcleo de negocio independiente de frameworks; puertos (interfaces) y adaptadores (implementaciones)	Arquitectura de software	Altamente testeable, desacople de infraestructura
Cliente/Servidor	Cliente solicita servicios a un servidor por red	Modelo de comunicación	Web moderna: front SPA + back API
SOA	Arquitectura orientada a servicios: servicios con contratos definidos; microservicios es su evolución	Arquitectura de software	Sistemas distribuidos con dominios funcionales bien delimitados

2) Cómo se conectan (niveles de abstracción)

- **Arquitectura de software:** decisiones macro (monolito, SOA/microservicios, hexagonal, etc.).
 - **Patrón de diseño arquitectónico:** plantillas de organización (MVC, por capas, microkernel, etc.).
 - **Patrón estructural:** cómo se disponen módulos/clases y sus relaciones (capas, adaptador, fachada, composite...).
 - **Modelo de comunicación:** cómo interactúan procesos/artefactos (cliente/servidor, pub/sub, request-reply).
- Capas puede verse tanto como **patrón estructural** (organiza dependencias) como **patrón arquitectónico** (estructura macro de la app).
- SOA existe:** es un estilo arquitectónico. **Microservicios** es su variante con servicios más pequeños/autónomos.

3) Comparación lado a lado (matriz)

Dimensión	Monolito	Capas	MVC	Hexagonal	Cliente/Servidor	SOA / Microservicios
Alcance	Arquitectura (artefacto único)	Patrón estructural/arquitectónico	Patrón arquitectónico (UI/server)	Arquitectura	Modelo de comunicación	Arquitectura
Despliegue	1 artefacto	Según arquitectura (puede ser 1)	Según arquitectura	1 artefacto (back) o por servicio	≥ 2 (front/back)	Varios servicios

Dimensión	Monolito	Capas	MVC	Hexagonal	Cliente/Servidor	SOA / Microservicios
Acoplamiento	Alto interno	Reduce acoplamiento por responsabilidades	Media (depende del framework)	Bajo entre dominio e infraestructura	Entre procesos	Bajo entre servicios (ideal)
Testabilidad	Media	Buena por capas	Buena en UI/server	Muy alta (dominio puro)	Independiente por lado	Alta por servicio
Escalabilidad	Replica el todo	Igual que arquitectura base	Igual que arquitectura base	Escala núcleos/port adapters	Escala lados por separado	Escala por servicio
Dónde va el "front"	Dentro del artefacto (SSR)	En capa de presentación	Parte "V" de MVC	Adaptador de entrada (UI/API)	Cliente separado	Cliente independiente por servicio (si aplica)
Persistencia	Interna	Capa de datos/infra	Dentro de "Model" (no solo DB)	Adaptador de salida (repo/DB)	En el servidor	Cada servicio gestiona su DB (ideal)
Pros	Simplicidad; time-to-market	Orden y separación	Claridad en UI	Desacople; testable; flexible	Claridad de límites	Autonomía; despliegues independientes
Contras	Crece y se vuelve rígido	Puede volverse burocrático	Confusiones "Model = DB"	Mayor diseño inicial	Gestión de contratos/API	Complejidad operativa (DevOps)
Cuándo usar	MVP, equipo pequeño	Necesitas orden y roles claros	UI web clásica	Lógica compleja con muchas integraciones	Front SPA + Back API	Dominios bien separados; equipos múltiples
Cuándo evitar	Dominios muy diversos	Equipo muy pequeño/prototipo	Backends sin vistas	Proyectos triviales	Apps puramente SSR	Equipos/infra sin madurez

4) Arquitectura ⇒ organización de paquetes (reglas y ejemplos)

Reglas prácticas

1. **Primero límites, luego carpetas** (dominios/módulos y dependencias permitidas).
2. **package-by-feature** sobre package-by-layer en monolitos medianos/grandes.
3. **Dominio y casos de uso no dependen de frameworks** (hexagonal/clean).
4. **DTO ≠ entidad de dominio**; contratos en **api/** (OpenAPI/Proto).
5. **Front y back separados** cuando adoptes cliente/servidor; compartir solo contratos.

Ejemplos

Go (backend hexagonal)

```

/internal
  /billing
    /domain
    /app
  /ports
  /in

```

```
/out
/adapters
/in/http
/out/db
/infra
/cmd/billing-api
```

Spring Boot (monolito por feature + capas)

```
src/main/java/com.acme
/users
/web
/app
/domain
/infra
/billing
/web | /app | /domain | /infra
```

Vue (cliente separado — cliente/servidor)

```
src/
/modules
/billing
  /views /components /store /services
/users
  /views /components /store /services
```

5) Rúbrica utilizada

Criterios

1. Precisión en entender el concepto.
2. Correspondencia con la realidad.
3. Ambigüedad (5 = nada ambiguo).

Ítem	Precisión	Correspondencia	Ambigüedad (5=claro)
Monolíticas	4	4	3
Capas	4	5	3
MVC	4	5	3
Hexagonal	4	5	2
Cliente/Servidor	4	5	2

Observaciones clave

- Monolito ≠ cliente/servidor: si el front es proyecto aparte, ya no es monolito.
- En MVC, **Model ≠ solo DB**; incluye reglas/estado de negocio.
- En hexagonal, la **UI es adaptador de entrada**; repos/ORM son adaptadores de salida.
- Cliente/servidor es un **modelo de comunicación**; el backend puede ser hexagonal, por capas, etc.

6) Gráfico de evaluación

 Gráfico de evaluación

Nota: mantener la imagen `architecture_rubric_scores.png` en la misma carpeta que este `.md` para visualizarla correctamente.

7) Checklist rápida para revisión de repos

- ☐ El dominio **no** importa infraestructura.
- ☐ Casos de uso expuestos como **puertos** (interfaces).
- ☐ Adaptadores implementan puertos y hacen mapeos.
- ☐ Features aisladas (carpetas por módulo).
- ☐ Contratos front/back en `api/` (no reutilizar entidades DB).
- ☐ Tests de dominio sin framework/DB.
- ☐ Documentar límites y dependencias permitidas en `ARCHITECTURE.md`.